

Projekt Dokumentáció: Kórus Weboldal

Ez a dokumentum a kórus weboldal technikai felépítését, választott megoldásait és követelményrendszerét részletezi.

1. Választott technológiai stack és indoklás

A projekt egy modern, háromrétegű webes architektúrára épül, amely szétválasztja a megjelenítési, az üzleti logika és az adatkezelési rétegeket.

Frontend: Angular

Az Angular keretrendszer választása a következő előnyök miatt történt:

- **Standalone komponensek:** A projekt az Angular legújabb funkcióit használja, ami egyszerűsíti a kódbázist és modulmentes fejlesztést tesz lehetővé.
- **Signals:** A reaktív állapotkezelés legmodernebb formája, amely biztosítja, hogy a felület azonnal frissüljön (pl. keresés vagy loading jelzések során).
- **Angular Material:** A Google tervezési elveit követő UI könyvtár biztosítja a professzionális megjelenést és a gyors fejlesztést a kész komponensekkel (Cards, Buttons, Inputs).
- **Angular Services:** Singleton objektumok, amelyek az alkalmazás teljes élettartama alatt élnek, és hidat képeznek a frontend és a backend között.
 - **AuthService:** Állapot- és token kezelésért felelős. JWT token valamint felhasználói adatok tárolása localStorage-ban.
 - **EventService:** Fellépésekkel kapcsolatos API hívások kezelése, backendről érkező adatok előkészítése a komponensek számára.
- **Angular Guards:** eldöntik, hogy a felhasználó beléphet-e egy adott útvonalra, mielőtt az egyáltalán betöltődne.
 - **AuthGuard:** Ha a felhasználó nincs bejelentkezve, automatikusan visszadobja a /login oldalra.

Backend: Node.js & Express

A backend réteg Node.js környezetben, Express keretrendszerrel készült, strukturált és moduláris felépítéssel:

- **REST API architektúra:** Tiszta interfészt biztosít az adatok lekéréséhez és módosításához.
- **MVC Architektúra:** A kód tiszta szétválasztása érdekében a rendszer a Model-View-Controller mintát követi, kiegészítve a Repository és Service rétegekkel.
- **Controller réteg:** Fogadja a HTTP kéréseket és meghívja a megfelelő service rétegbeli metódusokat.
- **Service réteg:** Itt dőlnek el az üzleti szabályok és itt történik a bonyolultabb logika kezelése (pl. jelszó hashelés, JWT generálás).

- **Model réteg:** Definiálja az adatstruktúrát (entity).
- **Repository réteg:** Az adatelérésért felelős réteg, amely elszigeteli az SQL lekérdezéseket a kód többi részétől, így az adatbázis később könnyen cserélhető.
- **View réteg:** A szerver json formában küldi el az adatokat a kliens számára, amely angular keretrendszert használva feldolgozza azokat.
- **Hitelesítés:** A rendszer JWT (JSON Web Token) alapú hitelesítést használ, a tokeneket localStorage-ban tároljuk a kliens oldalon.

Adatbázis: MySQL

Az események tárolására relációs adatbázist választottam:

- **Strukturált adatok:** Az események fix mezői (cím, dátum, helyszín) ideálisak a táblázatos szerkezethez.
- **Hatékonyság:** A kapcsolat kezelése Connection Pool segítségével történik.

2. Megvalósított funkcionális követelmények

A rendszer az alábbi üzleti funkciókat látja el:

Funkció	Leírás
Fellépések böngészése	Regisztrált felhasználók számára elérhető, esztétikus kártyás megjelenítés.
Globális keresés	Reaktív szűrés cím, helyszín, leírás és dátum mezőkre egyetlen beviteli mezővel.
Adminisztrátori CRUD	Események létrehozása, szerkesztése (meglévő adatok visszatöltésével) és törlése.
Hitelesítés	JWT alapú bejelentkezés és kijelentkezés a jogosultságok kezeléséhez.

3. Nem-funkcionális követelmények

A technikai minőséget biztosító szempontok:

- **Biztonság:** *AuthGuard* alkalmazása a védett útvonalakon. Ha nem hitelesített felhasználó próbál hozzáférni az admin funkciókhoz, a rendszer automatikusan a Login oldalra irányítja.
- **Válaszkészség (UX):**

- Interaktív betöltésjelzők (spinners) használata a hálózati kérések során.
- Input fókuszcra vagy eseményre aktiválódó reaktív frissítések.
- **Adatvédelem:** A jelszavak hashelt tárolása a backend oldalon és a tokenek biztonságos kezelése a kliens oldalon.